

Modeling

Introduction

What ⚡⚡

A *model* is a representation of something else.

📦 A class diagram is a model that represents a software design.

A *model* provides a simpler view of a complex entity because a model captures only a selected aspect. This omission of some aspects implies models are abstractions.

📦 A class diagram captures the structure of the software design but not the behavior.

Multiple models of the same entity may be needed to capture it fully.

📦 In addition to a class diagram (or even multiple class diagrams), a number of other diagrams may be needed to capture various interesting aspects of the software.

How ⚡⚡⚡

In software development, models are useful in several ways:

a) To analyze a complex entity related to software development.

📦 Some examples of using models for analysis:

1. Models of the problem domain can be built to aid the understanding of the problem to be solved.
2. When planning a software solution, models can be created to figure out how the solution is to be built. An architecture diagram is such a model.

b) To communicate information among stakeholders. Models can be used as a visual aid in discussions and documentation.

📦 Some examples of using models to communicate:

1. You can use an *architecture diagram* to explain the high-level design of the software to developers.
2. A business analyst can use a *use case diagram* to explain to the customer the functionality of the system.
3. A *class diagram* can be reverse-engineered from code so as to help explain the design of a component to a new developer.

c) As a blueprint for creating software. Models can be used as instructions for building software.

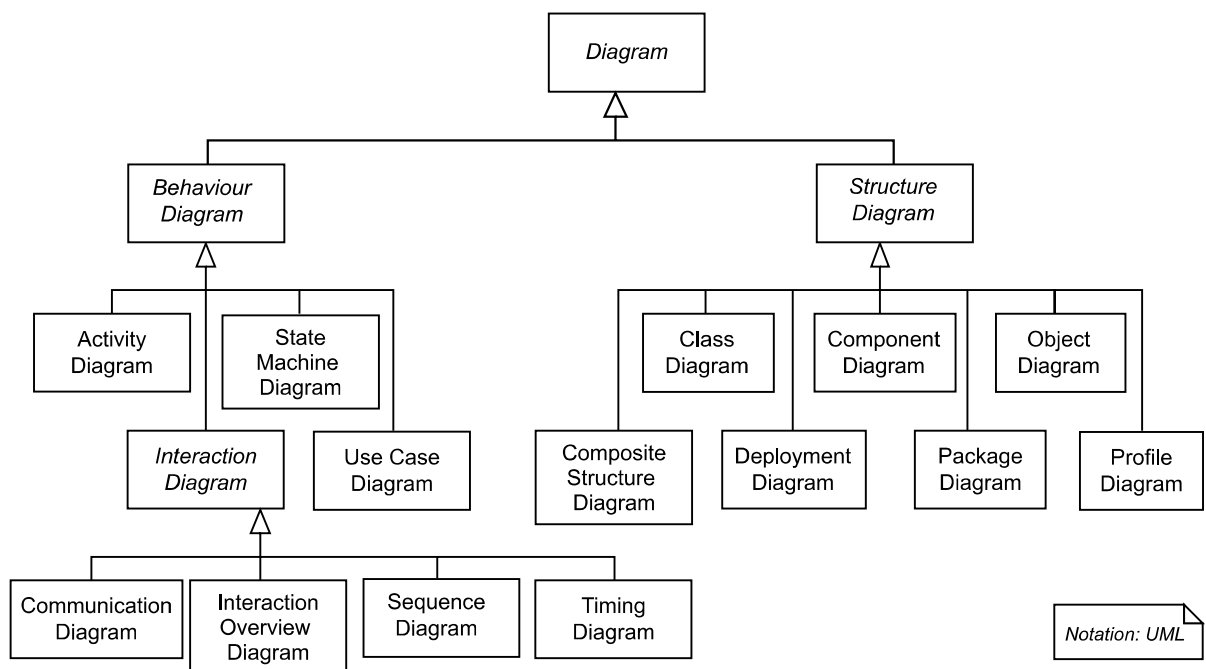
📦 Some examples of using models as blueprints:

1. A senior developer draws a class diagram to propose a design for an OOP software and passes it to a junior programmer to implement.
2. A software tool allows users to draw UML models using its interface and the tool automatically generates the code based on the model.

UML Models ⚡⚡⚡

 **Unified Modeling Language (UML)** is a graphical notation to describe various aspects of a software system. UML is the brainchild of three software modeling specialists James Rumbaugh, Grady Booch and Ivar Jacobson (also known as the Three Amigos). Each of them had developed their own notation for modeling software systems before joining forces to create a unified modeling language (hence, the term 'Unified' in UML). UML is currently the most commonly used modeling notation used in the software industry.

The following diagram uses the class diagram notation to show the different types of UML diagrams.



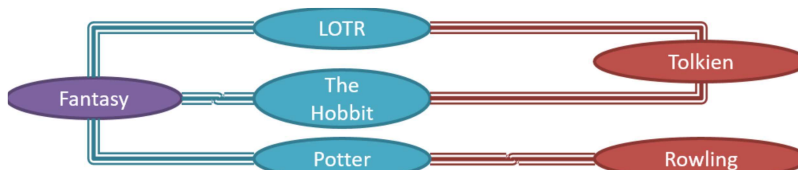
source: <https://en.wikipedia.org/>

Modeling structures

OO Structures ⚡

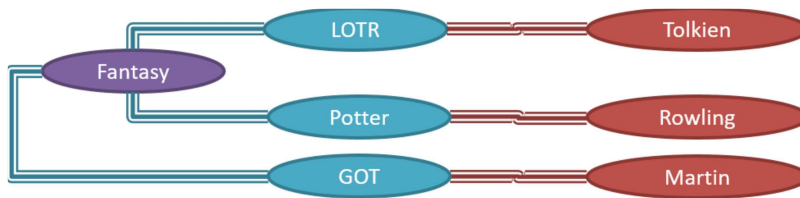
An OO solution is basically a network of objects interacting with each other. Therefore, **it is useful to be able to model how the relevant objects are 'networked' together** inside a software i.e. how the objects are connected together.

 Given below is an illustration of some objects and how they are connected together. Note: the diagram uses an ad-hoc notation.



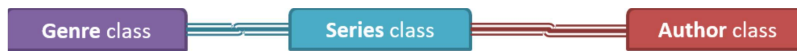
Note that these **object structures within the same software can change over time**.

Given below is how the object structure in the previous example could have looked like at a different time.



However, object structures do not change at random; they change based on a set of rules set by the designer of that software. Those **rules that object structures need to follow can be illustrated as a class structure** i.e. a structure that exists among the relevant classes.

Here is a class structure (drawn using an ad-hoc notation) that matches the object structures given in the previous two examples. For example, note how this class structure does not allow any connection between **Genre** objects and **Author** objects, a rule followed by the two object structures above.

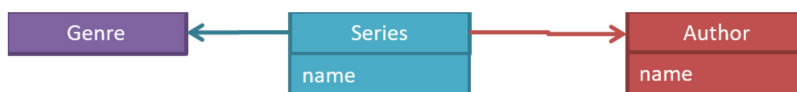


UML Object Diagrams model object structures. UML Class Diagrams model class structures.

Here is an object diagram for the above example:



And here is the class diagram for it:



Class Diagrams (Basics) ⚡

Contents related to UML diagrams in the panels given below belong to a different chapter (i.e., the chapter dedicated to UML); they have been embedded here for convenience.

Classes form the basis of class diagrams.

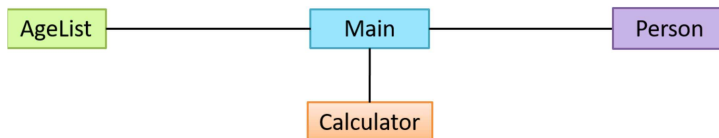
> 📖 UML ➡ Class Diagrams → Introduction → What

> 📖 UML ➡ Class Diagrams → Classes → What

Associations are the main connections among the classes in a class diagram.

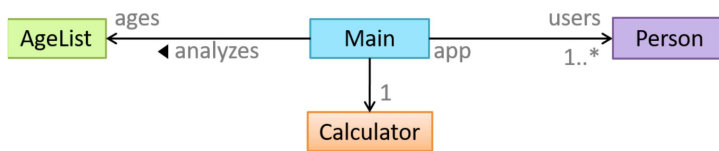
The most basic class diagram is a bunch of classes with some solid lines among them to represent associations, such as this one.

📦 An example class diagram showing associations between classes.



In addition, **associations can show additional decorations such as *association labels*, *association roles*, *multiplicity* and *navigability*** to add more information to a class diagram.

📦 Here is the same class diagram shown earlier but with some additional information included:



Adding More Info to UML Models ⚡⚡⚡

UML notes can be used to add more info to any UML model.

UML → Notes

✕
 ^

Loading...

^

Class Diagrams - Intermediate ⚡⚡

A class diagram can also show different types of relationships between classes: inheritance, compositions, aggregations, dependencies.

Modeling inheritance

OOP → Inheritance → What

The OOP concept *Inheritance* allows you to define a new class based on an existing class.

For example, you can use inheritance to define an `EvaluationReport` class based on an existing `Report` class so that the `EvaluationReport` class does not have to duplicate data/behaviors that are already implemented in the `Report` class. The `EvaluationReport` can inherit the `wordCount` attribute and the `print()` method from the *base class* `Report`.

- Other names for Base class: *Parent class*, *Superclass*
- Other names for Derived class: *Child class*, *Subclass*, *Extended class*

A superclass is said to be *more general* than the subclass. Conversely, a subclass is said to be more *specialized* than the superclass.

Applying inheritance on a group of similar classes can result in the common parts among classes being extracted into more general classes.

`Man` and `Woman` behave the same way for certain things. However, the two classes cannot be simply replaced with a more general class `Person` because of the need to distinguish between `Man` and `Woman` for certain other things. A solution is to add the `Person` class as a superclass (to contain the code common to men and women) and let `Man` and `Woman` inherit from `Person` class.

Inheritance implies the derived class can be considered as a *subtype* of the base class (and the base class is a *super-type* of the derived class), resulting in an *is a* relationship.

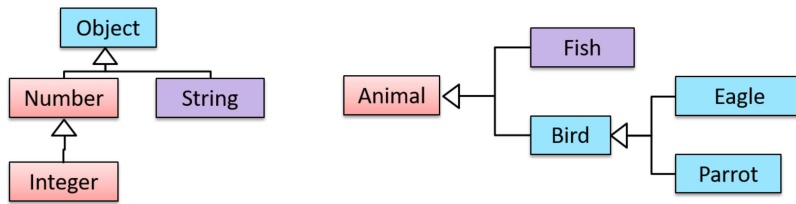
Inheritance does not necessarily mean a subtype relationship exists. However, the two often go hand-in-hand. For simplicity, at this point let us assume inheritance implies a subtype relationship.

To continue the previous example,

- `Woman` *is a* `Person`
- `Man` *is a* `Person`

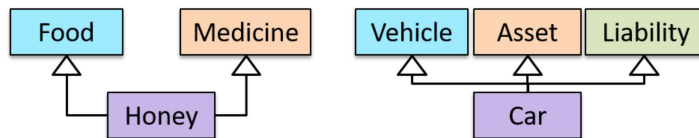
Inheritance relationships through a chain of classes can result in inheritance *hierarchies* (aka inheritance *trees*).

Two inheritance hierarchies/trees are given below. Note that the triangle points to the parent class. Observe how the **Parrot** is a **Bird** as well as it is an **Animal**.



Multiple Inheritance is when a class inherits *directly* from **multiple classes**. Multiple inheritance among classes is allowed in some languages (e.g., Python, C++) but not in other languages (e.g., Java, C#).

The **Honey** class inherits from the **Food** class and the **Medicine** class because honey can be consumed as a food as well as a medicine (in some oriental medicine practices). Similarly, a **Car** is a **Vehicle**, an **Asset** and a **Liability**.



Exercises:

Which statements are correct?



UML → Class Diagrams → Inheritance → What

Modeling composition

OOD → Associations → Composition

Loading...



UML → Class Diagrams → Composition → What

Modeling aggregation

OOD → Associations → Aggregation

Loading...



➤ 🎓 UML → Class Diagrams → Aggregation → What

Modeling dependencies

▼ 🎓 OOP → Associations → Dependencies

Loading...



➤ 🎓 UML → Class Diagrams → Dependencies → What

A class diagram can also show different types of class-like entities:

Modeling enumerations

▼ 🎓 OOP → Classes → Enumerations

Loading...



➤ 🎓 UML → Class Diagrams → Enumerations → What

Modeling abstract classes

▼ 🎓 OOP → Inheritance → Abstract Classes

Loading...



➤ 🎓 UML → Class Diagrams → Abstract Classes → What

Modeling interfaces

▼ 🎓 OOP → Inheritance → Interfaces

Loading...



➤ 🎓 UML → Class Diagrams → Interfaces → What

Class Diagrams - Advanced ⚡⚡⚡

A class diagram can show association classes too.

▼ 🎓 OOP → Associations → Association Classes

Loading...



➤ 🎓 UML → Class Diagrams → Association Classes → What

Object Diagrams ⚡

▼ 🎓 UML → Object Diagrams → Introduction

Loading...



Object diagrams can be used to complement class diagrams. For example, you can use object diagrams to model different object structures that can result from a design represented by a given class diagram.

▼ 🎓 UML → Object Diagrams → Objects

Loading...



▼ 🎓 UML → Object Diagrams → Associations

Loading...



Conceptual Class Diagrams (aka OODMs) ⚡⚡⚡

The analysis process for identifying objects and object classes is recognized as one of the most difficult areas of object-oriented development. .

-Ian Sommerville, in the book *Software Engineering*

Domain modeling is modeling the problem domain i.e. to model how things actually work in the real world. Domain modeling is useful in understanding the problem domain, which is essential to the success of a project.

Domain modeling can be done using,

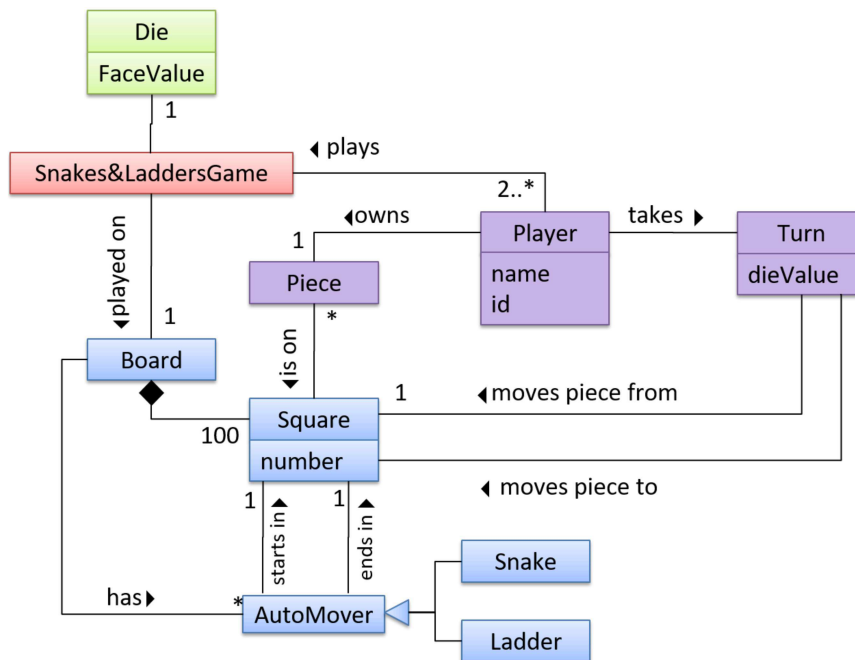
- a **domain-specific modeling notation** if such a notation exists (e.g., a modeling notation specific to the banking domain might have elements to represent loans, accounts, transactions etc.),
- or a **general purpose modeling notation**, such as UML (e.g., you can use an activity diagram to model the workflow of processing a loan application),
- or even other **general purpose notations** (e.g., you can use an organization chart to model the employee hierarchy of a company).

When building an OOP system, it makes sense to build OOP models of the problem domain, given OOP aspires to emulate the objects in the real world.

The UML model that captures class structures in the problem domain are called **conceptual class diagrams**. They are in fact a lighter version of class diagrams, and sometimes also called *OO domain models (OODMs)*. The latter name is somewhat misleading as conceptual class diagrams (CCDs) are actually only one type of domain models that can model an OOP problem domain.

The CCD of a snakes and ladders game is given below.

Description: The snakes and ladders game is played by two or more players using a board and a die. The board has 100 squares marked 1 to 100. Each player owns one piece. Players take turns to throw the die and advance their piece by the number of squares they earned from the die throw. The board has a number of snakes. If a player's piece lands on a square with a snake head, the piece is automatically moved to the square containing the snake's tail. Similarly, a piece can automatically move from a ladder foot to the ladder top. The player whose piece is the first to reach the 100th square wins.



CCDs do not contain solution-specific classes (i.e. classes that are used in the solution domain but do not exist in the problem domain). For example, a class called `DatabaseConnection` could appear in a class diagram but not usually in a CCD because `DatabaseConnection` is something related to a software solution but not an entity in the problem domain.

CCDs represents the class structure of the problem domain and not their behavior, just like class diagrams. To show behavior, use other diagrams such as sequence diagrams.

CCD notation is a subset of the class diagram notation (omits methods and navigability).

Modeling behaviors

Activity Diagrams - Basic ⚡⚡⚡

Software projects often involve workflows. Workflows define the flow in which a process or a set of tasks is executed. Understanding such workflows is important for the success of the software project.

Some examples in which a certain workflow is relevant to software project:

- 📦 A software that automates the work of an insurance company needs to take into account the workflow of processing an insurance claim.
- 📦 The algorithm of a piece of code represents the workflow (i.e. the execution flow) of the code.

❗ Contents related to UML diagrams in the panels given below belong to a different chapter (i.e., the chapter dedicated to UML); they have been embedded here in the Web version of the textbook (for convenience), but omitted in the printable version (to avoid repetition).

➤ 📖 UML ➡ Activity Diagrams → Introduction → What

➤ 📖 UML ➡ Activity Diagrams → Basic Notation → Linear Paths

➤ 📖 UML ➡ Activity Diagrams → Basic Notation → Alternate Paths

➤ 📖 UML ➡ Activity Diagrams → Basic Notation → Parallel Paths

Activity Diagrams - Intermediate ⚡⚡⚡

➤ 📖 UML ➡ Activity Diagrams → Intermediate Notation → Rakes

➤ 📖 UML ➡ Activity Diagrams → Intermediate Notation → Swim Lanes

Sequence Diagrams - Basic ⚡⚡

Sequence diagrams model the interactions between various entities in a system, in a specific scenario. Modelling such scenarios is useful, for example, to verify the design of the internal interactions is able to provide the expected outcomes.

Some examples where a sequence diagram can be used:

- 📦 To model how components of a system interact with each other to respond to a user action.
- 📦 To model how objects inside a component interact with each other to respond to a method call it received from another component.

❗ Contents related to UML diagrams in the panels given below belong to a different chapter (i.e., the chapter dedicated to UML); they have been embedded here for convenience.

➤ 📖 UML ➡ Sequence Diagrams → Introduction

➤  UML ➡ Sequence Diagrams → Basic Notation

➤  UML ➡ Sequence Diagrams → Loops

➤  UML ➡ Sequence Diagrams → Object Creation

➤  UML ➡ Sequence Diagrams → Minimal Notation

Sequence Diagrams - Intermediate ⚡⚡⚡

➤  UML ➡ Sequence Diagrams → Object Deletion

➤  UML ➡ Sequence Diagrams → Self-Invocation

➤  UML ➡ Sequence Diagrams → Alternative Paths

➤  UML ➡ Sequence Diagrams → Optional Paths

➤  UML ➡ Sequence Diagrams → Calls to Static Methods

Sequence Diagrams - Advanced ⚡⚡⚡

➤ UML: Sequence Diagrams: Parallel Paths

➤ UML: Sequence Diagrams: Reference Frames

Use Case Diagrams ⚡⚡⚡

Use case diagrams model the mapping between *features* of a system and its user *roles* i.e., which user roles can perform which tasks using the software.

 A simple use case diagram:



Modeling a solution

Introduction ⚡

You can use models to analyze and design software before you start coding.

Suppose you are planning to implement a simple minesweeper game that has a text based UI and a GUI. Given below is a possible OOP design for the game.



Before jumping into coding, you may want to find out things such as,

- Is this class structure able to produce the behavior you want?
- What API should each class have?
- Do you need more classes?

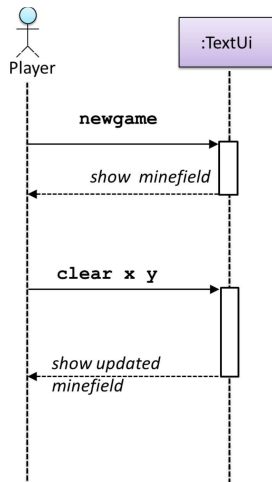
To answer these questions, you can analyze how the objects of these classes will interact with each other to produce the behavior you want.

Basic ⚡⚡

As mentioned in [Design → Modeling → Modeling a Solution → Introduction], this is the Minesweeper design you have come up with so far. Our objective is to analyze, evaluate, and refine that design.

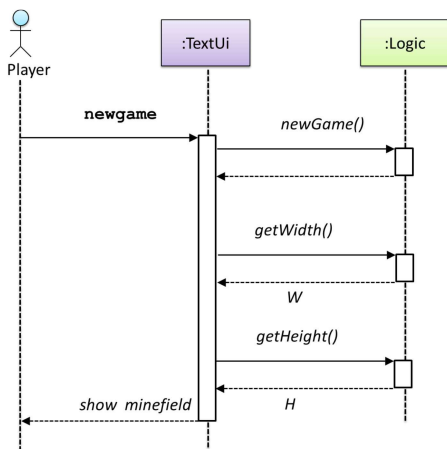


Let us start by modeling a sample interaction between the person playing the game and the `TextUi` object.



`newgame` and `clear x y` represent commands typed by the `Player` on the `TextUi`.

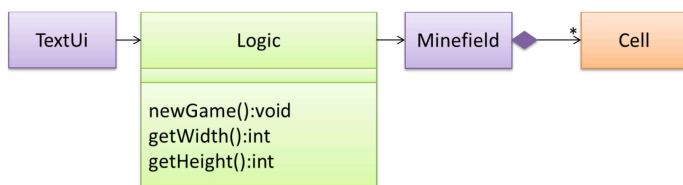
How does the `TextUi` object carry out the requests it has received from the player? It would need to interact with other objects of the system. Because the `Logic` class is the one that controls the game logic, the `TextUi` needs to collaborate with `Logic` to fulfill the `newgame` request. Let us extend the model to capture that interaction.



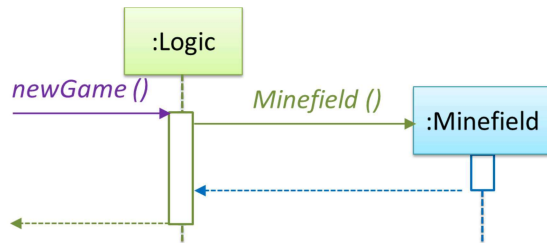
`W` = Width of the minefield; `H` = Height of the minefield

The above diagram assumes that `W` and `H` are the only information `TextUi` requires to display the minefield to the `Player`. Note that there could be other ways of doing this.

The `Logic` methods you conceptualized in our modeling so far are:

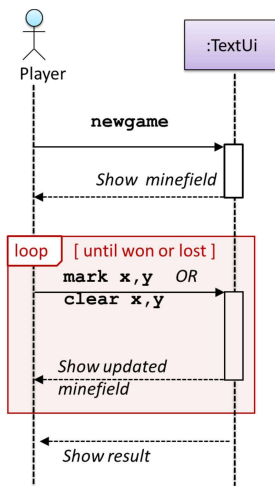


Now, let us look at what other objects and interactions are needed to support the `newGame()` operation. It is likely that a new `Minefield` object is created when the `newGame()` method is called.



Note that the behavior of the `Minefield` constructor has been abstracted away. It can be designed at a later stage.

Given below are the interactions between the player and the `TextUi` for the whole game.



💡 Note that a similar technique can be used when discovering/defining the architecture-level APIs.